

ICPC / IICPC C++ Contest Cheat Sheet

Syntax, pitfalls, templates — no explanations

1. Mandatory Boilerplate

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
}
```

2. STL Containers

- `vector<T> v;`
- `deque<T> dq;`
- `set<T>, multiset<T>`
- `unordered_set<T>`
- `map<K,V>, unordered_map<K,V>`
- `priority_queue<T>`
- `priority_queue<T, vector<T>, greater<T>>`

3. STL Operations

- `v.push_back(x), v.pop_back()`
- `v.size()` (**size_t**)
- `s.insert(x)`
- `s.erase(x)` vs `s.erase(it)`
- `mp[key]` creates key
- `mp.find(k) == mp.end()`
- `pq.top(), pq.pop()`

4. Sorting / Searching

```
sort(v.begin(), v.end());
sort(v.begin(), v.end(), greater<int>());
```

Custom comparator

```
bool cmp(const A& x, const A& y){
    if(x.a != y.a) return x.a < y.a;
    return x.b > y.b;
}
```

Binary search

- `lower_bound = first  $\geq x$`
- `upper_bound = first  $> x$`
- `index: it - v.begin()`

5. Fast I/O Traps

- Never mix `scanf` with `cin`
- Avoid `endl`
- Use `getline` for spaced strings

6. Integer Overflow

- Use `long long` for sums/products
- 10^{18} fits in `ll`

```
const ll INF = 4e18;
```

7. Modular Arithmetic

```
const int MOD = 1e9+7;
(a+b)%MOD
(a-b+MOD)%MOD
(a*b)%MOD
```

Modular exponentiation

```
ll modexp(ll a, ll e){
    ll r=1;
    while(e){
        if(e&1) r=r*a%MOD;
        a=a*a%MOD;
        e>>=1;
    }
    return r;
}
ll inv = modexp(a, MOD-2);
```

8. Graphs

Adjacency list

```
vector<vector<int>> adj(n);
adj[u].push_back(v);
```

BFS

```
queue<int> q;
vector<int> dist(n, -1);
dist[s]=0;
q.push(s);
while(!q.empty()){
    int u=q.front(); q.pop();
    for(int v:adj[u]){
        if(dist[v]==-1){
            dist[v]=dist[u]+1;
            q.push(v);
        }
    }
}
```

9. Dijkstra

```
priority_queue<pair<ll,int>,
vector<pair<ll,int>>, greater<>> pq;

pq.push({0,s});
while(!pq.empty()){
    auto [d,u]=pq.top(); pq.pop();
    if(d>dist[u]) continue;
```

```

for(auto [v,w]:adj[u]){
    if(dist[v]>d+w){
        dist[v]=d+w;
        pq.push({dist[v],v});
    }
}
}

```

10. DSU (Union-Find)

```

vector<int> p, sz;

int find(int x){
    return p[x]==x ? x : p[x]=find(p[x]);
}

void unite(int a,int b){
    a=find(a); b=find(b);
    if(a==b) return;
    if(sz[a]<sz[b]) swap(a,b);
    p[b]=a;
    sz[a]+=sz[b];
}

```

11. BIT (Fenwick)

```

// 1-indexed
for(; i<=n; i+=i&-i)

```

```

for(; i>0; i-=i&-i)

```

12. Strings

- 'a' vs "a"
- s.substr(pos, len)
- s.find(t) == string::npos
- stoi, stoll

13. Math Utilities

- __gcd(a,b)
- lcm = a/gcd(a,b)*b
- $\lceil a/b \rceil = (a+b-1)/b$

14. Debug

```

cerr << "x=" << x << "\n";

```

15. Common Fatal Errors

- 0-index vs 1-index mismatch
- Not clearing globals between testcases
- Overflow in distance / DP
- Forgetting continue in Dijkstra
- Using map when vector works

ICPC / Codeforces C++ Cheat Sheet (Personal)

Based on repeated failure modes — syntax only

0. Mandatory Boilerplate

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
}

```

1. Things You Mix Up (Quotes & Strings)

- 'a' is char, "a" is string
- char c; vs string s;
- cin >> s stops at space
- getline(cin, s) reads full line
- After cin >> x, call cin.ignore()

2. Indexing Rules (Write This First)

- Decide once: 0-indexed or 1-indexed

- Vectors: valid indices [0, n-1]
- Prefix sum:
 - pref[0]=0
 - pref[i]=sum of first i elements
- Ranges: [l, r) vs [l, r]

3. STL Containers (Usage Traps)

- map[key] **creates key**
- Use find() to test existence
- Prefer vector over map if keys are small
- unordered_map may TLE on hacks

4. Sorting & Comparators (You Forget This)

```

sort(v.begin(), v.end());
sort(v.begin(), v.end(), greater<int>());

```

Comparator must be strict

```

bool cmp(const A& x, const A& y){
    if(x.a != y.a) return x.a < y.a;
    return x.b > y.b;
}

```

5. Binary Search (Logic, Not Syntax)

- `lower_bound = first  $\geq x$`
- `upper_bound = first  $> x$`
- Equality check:
`it != v.end() && *it == x`
- Always verify monotonic condition

6. Integer Overflow (You Underestimate This)

- Use `ll` for:
 - distances
 - sums
 - DP values
- Cast before multiply

```
ll x = 1LL * a * b;
```

7. Modular Arithmetic (Negative Bug)

```
(a % MOD + MOD) % MOD
```

Never do

```
a = a - b % MOD;
```

8. Priority Queue Direction (You Ask This)

```
// Max heap (default)
priority_queue<int> pq;
```

```
// Min heap
priority_queue<int, vector<int>, greater<int>
>> pq;
```

9. Graph Algorithms (Stale State Bug)

Dijkstra

```
if(d > dist[u]) continue;
```

BFS Distance Init

```
vector<int> dist(n, -1);
```

Hash Maps / Dictionaries (Python & C++)

What it is: A hash map stores (key $\rightarrow$ value) pairs with average $O(1)$ insert, find, delete.

When to use:

- frequency counting
- mapping values to indices
- memoization

10. Multiple Testcases (You Forget Resets)

- Clear:
 - vectors
 - maps
 - adjacency lists
- Reinitialize globals
- Do NOT resize once and reuse blindly

11. Loop Bounds (Pattern / DP Errors)

- `for(i=0; i<n; i++)`
- `for(i=1; i<=n; i++)`
- Know which one you need
- DP transitions: check bounds **before** access

12. Common TLE Causes (You Hit These)

- map instead of vector
- $O(n^2)$ hidden in nested loops
- Recomputing prefix sums
- Sorting inside testcases unnecessarily

13. Math Utilities You Ask For

- `__gcd(a, b)`
- `lcm = a/gcd(a, b)*b`
- Ceiling division:
`(a+b-1)/b`

14. Debug (Remove Before Submit)

```
cerr << "x=" << x << "\n";
```

15. Final Mental Checklist

- Indexing consistent?
- Overflow possible?
- Reset per testcase?
- Correct container?
- Binary search condition valid?

- fast membership test (when set not enough)
-

Python: dict (YOU WILL USE THIS)

Creation

```
d = {}  
d = dict()
```

Insert / Update

```
d[key] = value
```

Access

```
x = d[key]           # KeyError if missing  
x = d.get(key, 0)    # safe
```

Check existence

```
if key in d:
```

Delete

```
del d[key]
```

Loop

```
for k in d:  
for k, v in d.items():
```

Frequency count (MOST COMMON USE)

```
from collections import Counter  
cnt = Counter(a)  
cnt[x]
```

Manual frequency

```
d = {}  
for x in a:  
    d[x] = d.get(x, 0) + 1
```

Python defaultdict (Cleaner)

```
from collections import defaultdict  
d = defaultdict(int)  
d[x] += 1
```

No need to check existence.

C++: unordered_map (For Comparison)

```
#include <unordered_map>  
unordered_map<int,int> mp;  
mp[key]++;           // creates key with 0  
if(mp.find(key) != mp.end())
```

WARNING:

- mp[key] creates key
 - Can TLE on hacks (worst-case $O(n)$)
-

What is a Collision? (Minimal Theory)

Hash map uses:

$$\text{index} = h(\text{key}) \bmod N$$

A **collision** occurs when:

$$h(k_1) \equiv h(k_2) \pmod{N}, \quad k_1 \neq k_2$$

Multiple keys map to same bucket.

Resolution methods (conceptual):

- chaining (linked list / array)
- open addressing

IMPORTANT:

- Python handles collisions internally
 - You do **NOT** manage collisions manually
-

Complexity Guarantees (Contest-Relevant)

- Average case: $O(1)$
- Worst case: $O(n)$ (pathological collisions)

In contests:

- Python dict is safe for CF/IICPC
 - C++ `unordered_map` can be hacked
 - If keys are small $\rightarrow$ use vector/array
-

Common Mistakes YOU Must Avoid

- Assuming sorted order (dict is insertion-ordered only)
 - Using dict when array suffices
 - Forgetting `get()`
 - Modifying dict while iterating
 - Using floating-point keys
-

When NOT to Use Hash Maps

- keys in range $[0, n] \rightarrow$ use array
 - need ordered keys $\rightarrow$ sort or use tree
 - tight memory constraints
-

Mental Checklist (Before Using dict)

- Are keys integers?
- Is order irrelevant?
- Can an array replace this?
- Is frequency counting needed?

Standard CSES Patterns (Thinking Templates)

These are NOT full solutions. They are minimal skeletons to recognize the pattern and fill logic.

1. BFS (Unweighted Shortest Path)

Used in:

- Labyrinth
- Message Route
- Monsters

```
vector<vector<int>>> adj(n);
vector<int> dist(n, -1);
queue<int> q;

dist[s] = 0;
q.push(s);

while(!q.empty()){
    int u = q.front(); q.pop();
    for(int v : adj[u]){
        if(dist[v] == -1){
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
}
```

2. DFS (Connected Components)

Used in:

- Counting Rooms
- Building Roads

```
vector<vector<int>>> adj(n);
vector<bool> vis(n,false);

void dfs(int u){
    vis[u] = true;
    for(int v : adj[u]){
        if(!vis[v]) dfs(v);
    }
}
```

Mental cue: **count components / flood fill**

3. Dijkstra (Weighted Graph)

Used in:

- Shortest Routes I
- Flight Discount

```
const ll INF = 4e18;
vector<vector<pair<int,ll>>> adj(n);
vector<ll> dist(n, INF);

priority_queue<pair<ll,int>,
vector<pair<ll,int>>, greater<>> pq;

dist[s] = 0;
pq.push({0, s});

while(!pq.empty()){
    auto [d,u] = pq.top(); pq.pop();
    if(d > dist[u]) continue;

    for(auto [v,w] : adj[u]){
        if(dist[v] > d + w){
            dist[v] = d + w;
            pq.push({dist[v], v});
        }
    }
}
```

Mental cue: **positive weights + shortest path**

4. Topological Sort (DAG)

Used in:

- Course Schedule
- Game Routes

```
vector<vector<int>>> adj(n);
vector<int> indeg(n,0);
queue<int> q;

for(int u=0;u<n;u++)
    for(int v:adj[u]) indeg[v]++;

for(int i=0;i<n;i++)
    if(indeg[i]==0) q.push(i);

while(!q.empty()){
```

```

    int u=q.front(); q.pop();
    for(int v:adj[u]){
        if(--indeg[v]==0)
            q.push(v);
    }
}

```

Mental cue: **DAG + order matters**

5. DSU / Union-Find

Used in:

- Road Construction
- Road Reparation

```

vector<int> p, sz;

int find(int x){
    return p[x]==x ? x : p[x]=find(p[x]);
}

void unite(int a,int b){
    a=find(a); b=find(b);
    if(a==b) return;
    if(sz[a]<sz[b]) swap(a,b);
    p[b]=a;
    sz[a]+=sz[b];
}

```

Mental cue: **dynamic connectivity**

6. Grid DFS / BFS

Used in:

- Labyrinth
- Counting Rooms

```

int dx[4]={1,-1,0,0};
int dy[4]={0,0,1,-1};

void dfs(int x,int y){
    vis[x][y]=true;
    for(int d=0;d<4;d++){
        int nx=x+dx[d], ny=y+dy[d];
        if(valid(nx,ny) && !vis[nx][ny])
            dfs(nx,ny);
    }
}

```

Mental cue: **2D movement**

7. Prefix Sums

Used in:

- Static Range Sum Queries
- Forest Queries

```

vector<ll> pref(n+1,0);
for(int i=0;i<n;i++)
    pref[i+1] = pref[i] + a[i];

sum(l,r) = pref[r] - pref[l];

```

Mental cue: **many queries, static array**

8. DP on 1D Array

Used in:

- Dice Combinations
- Minimizing Coins

```
vector<ll> dp(n+1, INF);
dp[0] = 0;

for(int i=1; i<=n; i++){
    for(int c : coins){
        if(i-c >= 0)
            dp[i] = min(dp[i], dp[i-c] + 1);
    }
}
```

Mental cue: **state = prefix**

9. Longest Path in DAG

Used in:

- Longest Flight Route

```
vector<ll> dp(n, -INF);
dp[s]=0;

for(int u : topo){
    if(dp[u] < 0) continue;
    for(int v:adj[u]){
        dp[v] = max(dp[v], dp[u] + 1);
    }
}
```

Mental cue: **topo + DP**

10. Cycle Detection (Directed)

```
vector<int> state(n,0); // 0=unvisited,1=visiting,2=done

bool dfs(int u){
    state[u]=1;
    for(int v:adj[u]){
        if(state[v]==1) return true;
        if(state[v]==0 && dfs(v)) return true;
    }
    state[u]=2;
    return false;
}
```

Mental cue: **detect impossible ordering**